

WEEK – 10

NAME:CH.SAI VIHAMSH

ROLL:2520090090

SESSION-19

Task1: To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

```
saivihamsh@Varshini-PC:~$ cd ~/OSSP
saivihamsh@Varshini-PC:~/OSSP$ mkdir -p skill/week10/task1
saivihamsh@Varshini-PC:~/OSSP$ cd skill/week10/task1
saivihamsh@Varshini-PC:~/OSSP/skill/week10/task1$ nano signal_handlers.c
saivihamsh@Varshini-PC:~/OSSP/skill/week10/task1$ gcc signal_handlers.c -
o T1
saivihamsh@Varshini-PC:~/OSSP/skill/week10/task1$ ./T1
```

```
=====
          SKILL WEEK 10 - TASK 1
          SIGNAL HANDLER TEST
=====
Process PID : 791
Process PGID: 791
```

Signal handlers registered successfully.

Supported signals:
SIGINT - Interrupt signal
SIGUSR1 - User-defined signal
SIGTERM - Termination signal

Testing signal events...
Press Ctrl+C to test SIGINT.
Or use another terminal to send SIGUSR1.

Waiting for signals...

^C

Signal Event Detected

Signal : SIGINT
Sender PID : 0
Signal Count : 1

Signal Event Detected

Signal : SIGUSR1
Sender PID : 804
Signal Count : 2

Signal Event Detected

Signal : SIGTERM
Sender PID : 804
Signal Count : 3

Required signal events received.
Signal handling test completed successfully.

```

GNU nano 8.7.1 signal_handlers.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t signal_count = 0;

/* Signal handler */
void signal_handler(int sig, siginfo_t *info, void *context)
{
    signal_count++;

    printf("\n-----\n");
    printf("Signal Event Detected\n");
    printf("-----\n");

    if (sig == SIGINT)
        printf("Signal      : SIGINT\n");
    else if (sig == SIGUSR1)
        printf("Signal      : SIGUSR1\n");
    else if (sig == SIGTERM)
        printf("Signal      : SIGTERM\n");
    else
        printf("Signal Number: %d\n", sig);

    printf("Sender PID   : %d\n", info->si_pid);
    printf("Signal Count : %d\n", signal_count);
    printf("-----\n");
}

/* Register signal handlers */
void register_handlers()
{
    struct sigaction action;

    memset(&action, 0, sizeof(action));

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify

```

```

GNU nano 8.7.1 signal_handlers.c *
}

int main()
{
    printf("=====\n");
    printf("      SKILL WEEK 10 - TASK 1\n");
    printf("      SIGNAL HANDLER TEST\n");
    printf("=====\n");

    printf("Process PID : %d\n", getpid());
    printf("Process PGID: %d\n", getpgrp());

    register_handlers();

    printf("\nSignal handlers registered successfully.\n");

    printf("\nSupported signals:\n");
    printf("SIGINT - Interrupt signal\n");
    printf("SIGUSR1 - User-defined signal\n");
    printf("SIGTERM - Termination signal\n");

    printf("\nTesting signal events...\n");
    printf("Press Ctrl+C to test SIGINT.\n");
    printf("Or use another terminal to send SIGUSR1.\n");
    printf("\nWaiting for signals...\n");

    while (signal_count < 3) {
        pause();
    }

    printf("\nRequired signal events received.\n");
    printf("Signal handling test completed successfully.\n");

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify

```

Observation: The signal handling program was successfully implemented using signal handlers and `sigaction()`. The program registered handlers for `SIGINT`, `SIGUSR1`, and `SIGTERM` and successfully received the generated signals. `SIGINT` was tested using `Ctrl+C`, while `SIGUSR1` and `SIGTERM` were sent to the process using its PID. The program displayed the signal information and maintained a signal count, demonstrating correct signal delivery, signal context management, and stable signal handling.

Task2: To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

```
saivihamsh@Varshini-PC:~/OSSP/skill/week10/task1$ cd ~/OSSP/skill/week10
saivihamsh@Varshini-PC:~/OSSP/skill/week10$ mkdir -p task2
saivihamsh@Varshini-PC:~/OSSP/skill/week10$ cd task2
saivihamsh@Varshini-PC:~/OSSP/skill/week10/task2$ nano sigint_forward.c
saivihamsh@Varshini-PC:~/OSSP/skill/week10/task2$ gcc sigint_forward.c -o
T2
saivihamsh@Varshini-PC:~/OSSP/skill/week10/task2$ ./T2
=====
          SKILL WEEK 10 - TASK 2
          SIGINT FORWARDING SYSTEM
=====
Shell PID : 1414
Shell PGID: 1414

Shell SIGINT handler registered.
Shell is protected from direct termination.

===== MENU =====
1. Start Foreground Job
2. Display Job State
3. Exit Shell
=====
Enter your choice: 1

Foreground job started successfully.
PID   : 1415
State : Running

Foreground child started.
Child PID: 1415
Foreground job is running...
Press Ctrl+C to interrupt the foreground job.
Job executing... 1/20
Job executing... 2/20
Job executing... 3/20
Job executing... 4/20
Job executing... 5/20
Job executing... 6/20
Job executing... 7/20
Job executing... 8/20
Job executing... 9/20
```

```

Foreground Job State: Terminated (PID: 0)

===== MENU =====
1. Start Foreground Job
2. Display Job State
3. Exit Shell
=====
Enter your choice: 1

Foreground job started successfully.
PID   : 1423
State : Running

Foreground child started.
Child PID: 1423
Foreground job is running...
Press Ctrl+C to interrupt the foreground job.
Job executing... 1/20
Job executing... 2/20
Job executing... 3/20
Job executing... 4/20
Job executing... 5/20
Job executing... 6/20
Job executing... 7/20
Job executing... 8/20
^C

Shell received SIGINT.
Forwarding SIGINT to foreground job PID 1423...

Foreground job was terminated by signal 2.
Updated Job State: Terminated

===== MENU =====
1. Start Foreground Job
2. Display Job State
3. Exit Shell
=====
Enter your choice: 3
Exiting shell.

```

```

GNU nano 8.7.1 sigint_forward.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

typedef enum {
    RUNNING,
    COMPLETED,
    TERMINATED
} JobState;

typedef struct {
    pid_t pid;
    JobState state;
} Job;

Job foreground_job = {0, COMPLETED};
volatile sig_atomic_t child_finished = 0;

/* Display current job state */
void display_state()
{
    if (foreground_job.state == RUNNING)
        printf("Running");
    else if (foreground_job.state == COMPLETED)
        printf("Completed");
    else
        printf("Terminated");
}

/* Capture SIGINT in the shell and forward it */
void sigint_handler(int sig)
{
    printf("\n\nShell received SIGINT.\n");

    if (foreground_job.pid > 0 &&

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```
GNU nano 8.7.1 sigint_forward.c *
printf("3. Exit Shell\n");
printf("=====\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
    case 1:
        run_foreground_job();
        break;

    case 2:
        printf("\nForeground Job State: ");

        if (foreground_job.pid == 0 &&
            foreground_job.state == COMPLETED) {
            printf("No active foreground job\n");
        } else {
            display_state();
            printf(" (PID: %d)\n",
                foreground_job.pid);
        }
        break;

    case 3:
        printf("Exiting shell.\n");
        return 0;

    default:
        printf("Invalid choice.\n");
}

return 0;
}
```

GNU nano 8.7.1 **sigint_forward.c ***

^G Help **^O** Write Out **^F** Where Is **^K** Cut **^T** Execute
^X Exit **^R** Read File **^_** Replace **^U** Paste **^J** Justify

Observation: The SIGINT forwarding system was successfully implemented using Linux signal handling and process management functions. The shell registered a SIGINT handler and remained protected from direct termination while a foreground process was running. When Ctrl+C was pressed, the shell captured the SIGINT signal and forwarded it to the foreground job, causing the foreground process to terminate while the shell continued running. The job state was then updated to Terminated, and normal foreground execution was also verified by allowing another job to complete successfully. Thus, interrupt handling, signal forwarding, foreground job termination, and job state management were successfully demonstrated.

SESSION-20

Task1: To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

```
Exiting shell.
saivihamsh@Varshini-PC:~/OSSP/skill/week10/task2$ cd ~/OSSP/skill/week10
saivihamsh@Varshini-PC:~/OSSP/skill/week10$ mkdir -p task1
saivihamsh@Varshini-PC:~/OSSP/skill/week10$ cd task1
saivihamsh@Varshini-PC:~/OSSP/skill/week10/task1$ nano sigtstp_job.c
saivihamsh@Varshini-PC:~/OSSP/skill/week10/task1$ gcc sigtstp_job.c -o T1
saivihamsh@Varshini-PC:~/OSSP/skill/week10/task1$ ./T1
```

```
=====
          SKILL WEEK 10 - TASK 1
          SIGTSTP JOB CONTROL
=====

===== MENU =====
1. Start Foreground Job
2. Resume Stopped Job
3. Display Job Table
4. Exit
=====
Enter your choice: 1

=====
Job Started
=====
Job ID : 1
PID    : 1552
State  : Running

Foreground job started.
Child PID: 1552
Job is running...
Job executing... 1/30
Job executing... 2/30
Job executing... 3/30
Job executing... 4/30
Job executing... 5/30
^Z
SIGTSTP received.
Suspending foreground job PID 1552...
Job 1 is now stopped.

Job was suspended.
```

```
=====
Enter your choice: 2

Resuming job 1...
SIGCONT sent successfully.
Job State: Running
Job executing... 6/30
Job executing... 7/30
Job executing... 8/30
Job executing... 9/30
Job executing... 10/30
Job executing... 11/30
Job executing... 12/30
Job executing... 13/30
Job executing... 14/30
Job executing... 15/30
Job executing... 16/30
Job executing... 17/30
Job executing... 18/30
Job executing... 19/30
^Z
SIGTSTP received.
Suspending foreground job PID 1552...
Job 1 is now stopped.

===== MENU =====
1. Start Foreground Job
2. Resume Stopped Job
3. Display Job Table
4. Exit
=====
Enter your choice: 4

Exiting program.
```

```

GNU nano 8.7.1                                sigtstp_job.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

typedef enum {
    RUNNING,
    STOPPED,
    COMPLETED,
    TERMINATED
} JobState;

typedef struct {
    int job_id;
    pid_t pid;
    JobState state;
} Job;

Job foreground_job = {1, 0, COMPLETED};

void display_state()
{
    switch (foreground_job.state) {
        case RUNNING:
            printf("Running");
            break;

        case STOPPED:
            printf("Stopped");
            break;

        case COMPLETED:
            printf("Completed");
            break;

        case TERMINATED:
    
```

^{^G} Help ^{^O} Write Out ^{^F} Where Is ^{^K} Cut ^{^T} Execute
^{^X} Exit ^{^R} Read File ^{^_\} Replace ^{^U} Paste ^{^J} Justify

```

GNU nano 8.7.1                                sigtstp_job.c *

    printf("\n===== MENU =====\n");
    printf("1. Start Foreground Job\n");
    printf("2. Resume Stopped Job\n");
    printf("3. Display Job Table\n");
    printf("4. Exit\n");
    printf("===== \n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            run_job();
            break;

        case 2:
            resume_job();
            break;

        case 3:
            display_job();
            break;

        case 4:
            printf("\nExiting program.\n");
            return 0;

        default:
            printf("\nInvalid choice.\n");
    }
}

return 0;
}

```

Observation: The SIGTSTP job control system was successfully implemented using Linux signal handling and process management functions. The program captured the SIGTSTP signal and suspended the running foreground process. The job table was updated to preserve the job ID, process ID, and Stopped state. The stopped job was then resumed using the SIGCONT signal, after which its state changed to Running and the job continued until completion. Thus, the suspension, state preservation, job resumption, and completion workflow were successfully demonstrated.

Task2: Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

```
saivihamsh@Varshini-PC:~/OSSP/skill/week10/task2$ nano process_groups.c
saivihamsh@Varshini-PC:~/OSSP/skill/week10/task2$ gcc process_groups.c -o
T2
saivihamsh@Varshini-PC:~/OSSP/skill/week10/task2$ ./T2
=====
SKILL WEEK 10 - TASK 2
PROCESS GROUP CONTROL
=====
Shell PID   : 1743
Shell PGID  : 1743

=====
Process Group Information
=====
Parent PID  : 1743
Parent PGID : 1743
Child PID   : 1744
Child PGID  : 1744

Process group created successfully.

Transferring terminal control to child process group...
Terminal control transferred successfully.
Child process group now owns the terminal.
```

```
=====
Child Process Group Created
=====
Child PID   : 1744
Child PGID  : 1744

Child process is running...
Child executing... 1/10
Child executing... 2/10
Child executing... 3/10
Child executing... 4/10
Child executing... 5/10
Child executing... 6/10
Child executing... 7/10
Child executing... 8/10
Child executing... 9/10
Child executing... 10/10

Child process completed.

[2]+  Stopped                  ./T2
```

```

GNU nano 8.7.1 process_groups.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

pid_t child_pid = -1;
pid_t child_pgid = -1;

void display_process_info(const char *name, pid_t pid)
{
    printf("\n%s\n", name);
    printf("PID : %d\n", pid);
    printf("PGID : %d\n", getpgid(pid));
}

int main()
{
    int status;
    pid_t shell_pid = getpid();
    pid_t shell_pgid = getpgrp();

    printf("=====\n");
    printf("          SKILL WEEK 10 - TASK 2\n");
    printf("          PROCESS GROUP CONTROL\n");
    printf("=====\n");

    printf("Shell PID : %d\n", shell_pid);
    printf("Shell PGID : %d\n", shell_pgid);

    child_pid = fork();

    if (child_pid < 0) {
        perror("fork");
        return 1;
    }

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify

```

```

GNU nano 8.7.1 process_groups.c *
    printf("Child PID : %d\n", child_pid);
    printf("Child PGID : %d\n", getpgid(child_pid));

    printf("\nProcess group created successfully.\n");

    printf("\nTransferring terminal control to child process group...\n");

    if (tcsetpgrp(STDIN_FILENO, child_pgid) == -1) {
        perror("tcsetpgrp");
    } else {
        printf("Terminal control transferred successfully.\n");
    }

    printf("\nChild process group now owns the terminal.\n");

    sleep(2);

    printf("\nRestoring terminal control to parent...\n");

    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1) {
        perror("tcsetpgrp");
    } else {
        printf("Terminal ownership restored to parent.\n");
    }

    waitpid(child_pid, &status, 0);

    printf("\n=====\n");
    printf("Process Group Test Completed\n");
    printf("=====\n");

    if (WIFEXITED(status)) {
        printf("Child exited normally.\n");
        printf("Exit status: %d\n", WEXITSTATUS(status));
    } else if (WIFSIGNALED(status)) {
        printf("Child terminated by signal %d.\n",
            WTERMSIG(status));
    }

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify

```

Observation: The process group control system was successfully implemented using Linux process management and terminal control functions. A separate process group was created for the child process, and the process ID and process group ID were successfully displayed. Terminal control was transferred to the child process group and later restored to the parent process after the child completed execution. The child process executed successfully and exited normally, demonstrating proper process group creation, terminal ownership transfer, restoration, and process coordination.